Chapter 1 Proposed Method

In this chapter, we present the proposed integration automation framework designed to streamline and accelerate the end-to-end (E2E) testing process of O-RAN-based systems. Building upon the challenges identified in the previous chapter—such as the need to manually configure each network component, verify synchronization status, and confirm UE connectivity—we propose a solution that automates the configuration, deployment, monitoring, and validation steps across all relevant nodes, including the 5GC, gNB, RU, and UE.

The goal of this framework is to reduce the manual effort required during integration testing, minimize human errors caused by inconsistent configurations, and improve the repeatability and efficiency of system validation. The proposed method leverages remote command execution, configuration file generation, traffic and log analysis, and structured test orchestration logic to create a unified, end-to-end automated testing workflow. As illustrated in Fig. 1.2, the overall system working flow integrates configuration, deployment, monitoring, and validation into a unified automated process.

1.1 System Input Parameters for rApp Execution

In this system, users are required to provide three categories of input parameters as prerequisites for executing the rApp: **gNB Configuration**,

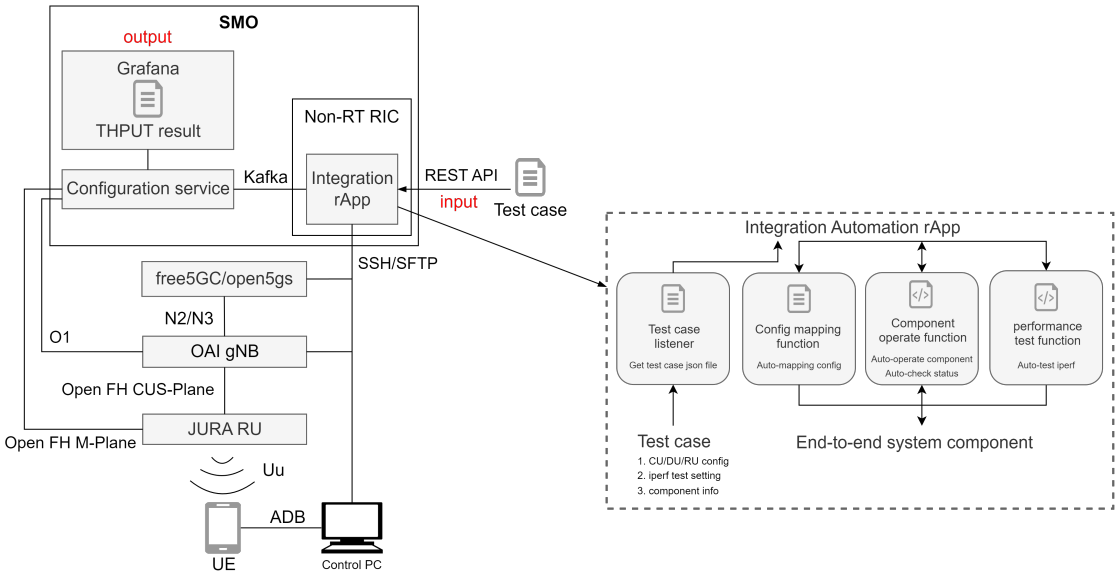

Figure 1.1: Proposed architecture

Test Configuration, and **Host Information**. These parameters are parsed by the automation testing platform and used for remote device configuration, core network initialization, gNB launching, downlink/uplink traffic testing, synchronization monitoring, and more. Accurate and consistent parameter settings are essential to ensure interoperability between DU and RU and the correctness of automated testing procedures.

As shown in the system working flow in Figure 1.3, the first step of the rApp is to wait for and receive configuration input from the user. The configuration is provided in JSON format and must include all required fields described in the following tables. Once received, the rApp parses the configuration file and proceeds with sequential automation steps such as DU/RU configuration, gNB and core network startup, and performance or synchronization verification.

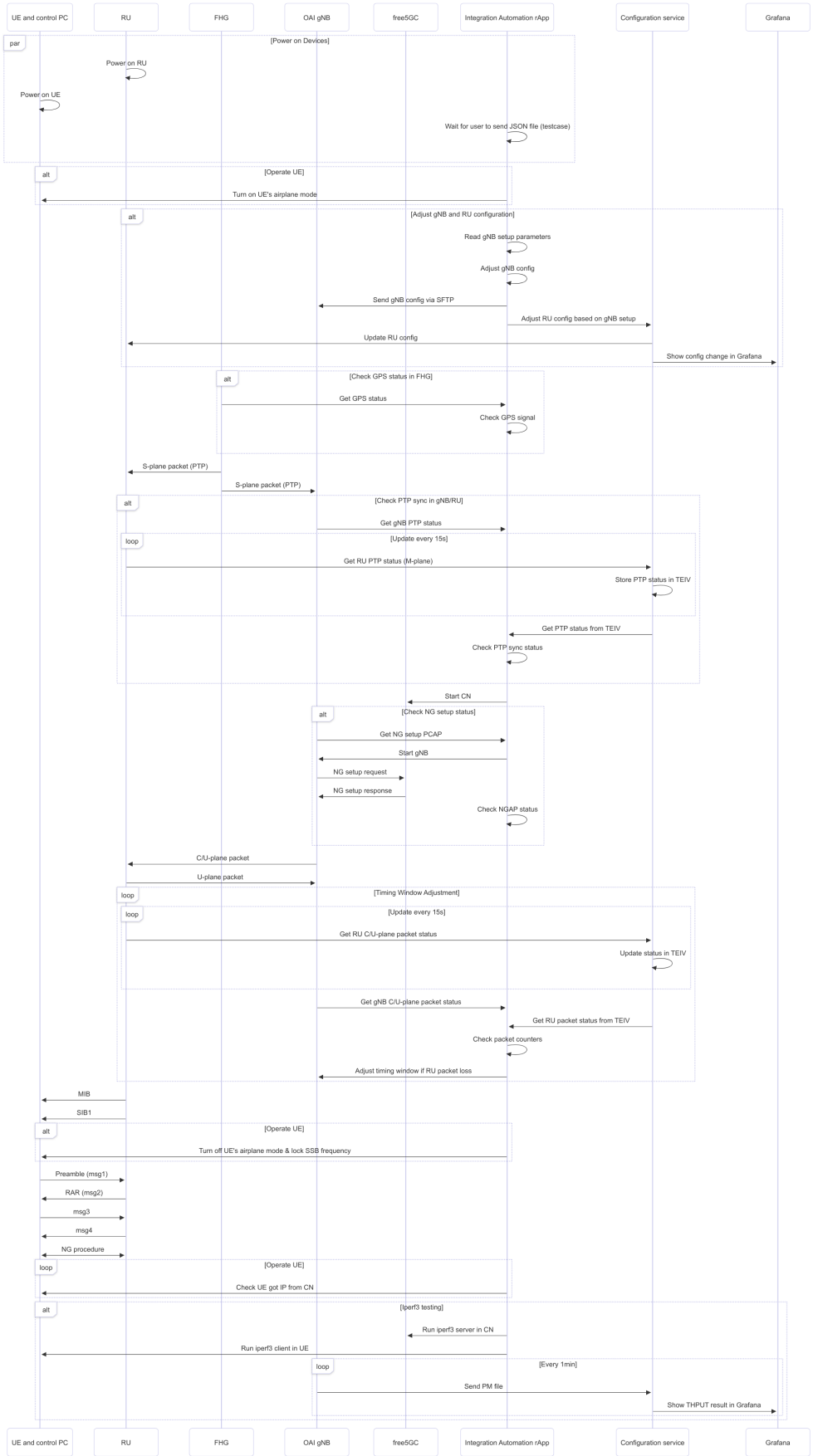

Figure 1.2: Working flow of integration automation

Table 1.1: gNB Configuration Parameters

Parameter	**Description**
core_id	Core Network Identifier
du_id	Distributed Unit Identifier
ru_id	Radio Unit Identifier
dpdk_devices_interface	Network interface used by DPDK
vlan-id	VLAN ID for fronthaul connection
o-du-mac-address	MAC address of DU interface
l2-mtu	Layer 2 Maximum Transmission Unit
Antenna config	eAxC ID list and phisical antenna setting
iq-bitwidth	Bit width of IQ samples
compression-type	Compression type for IQ data (e.g., STATIC)
compression-method	Compression algorithm (e.g., BLOCK_FLOATING_POINT)
gain	Transmission gain
arfcn	Absolute Radio Frequency Channel Number
bSChannelBw	Channel bandwidth in MHz
subCarrierSpacing	Subcarrier spacing in kHz
maxMIMO_layers	Max number of MIMO transmission layers
mcc	Mobile Country Code
mnc	Mobile Network Code
amf_ip_address	AMF (Core Network) IP address
GNB_IPV4_ADDRESS_FOR_NG_AMF	gNB IP address for N2/N3 interface

Table 1.2: Test Configuration Parameters

Parameter	**Description**
STOP_GNB	Whether to stop gNB before starting the test (yes/no)
ENABLE_DL	Enable downlink testing
ENABLE_UL	Enable uplink testing
TEST_DURATION	Duration of the test in seconds
DL_START	Time to start DL traffic in ms
UL_START	Time to start UL traffic in ms
TEST_SERVER_IP	IP address of the test server
ADB_DEVICE	Android ADB device ID
PTP4L_SERVICE	PTP synchronization service name
PHC2SYS_SERVICE	PHC to system time sync service
GNB_BASE_PATH	File system path to gNB source directory
O1_BASE_PATH	File system path to O1 adapter
CN_BASE_PATH	File system path to Core Network source
CN_RUN_COMMAND	Script to run the Core Network
CN_STOP_COMMAND	Script to stop the Core Network

Table 1.3: Host Information Parameters

Parameter	**Definition**
CN_SERVER_USER	Username to log in to core network server
CN_SERVER_HOST	IP address of core network server
CN_SERVER_PASSWORD	Password for CN server access
GNB_SERVER_USER	Username to log in to gNB server
GNB_SERVER_HOST	IP address of gNB server
GNB_SERVER_PASSWORD	Password for gNB server access
FHG_SERVER_USER	Username for fronthaul gateway server
FHG_SERVER_HOST	IP address of fronthaul gateway server
FHG_SERVER_PASSWORD	Password for fronthaul gateway access
CONTROL_PC_USER	Username for control PC
CONTROL_PC_HOST	IP address of control PC
CONTROL_PC_PASSWORD	Password for control PC access

1.2 Automatic Configuration mapping of gNB and RU

As shown in Figure 1.3, the system executes the following automatic configuration mapping steps across the CU/DU and RU:

- Read gNB setup parameters from the user JSON input.
- Adjust the gNB configuration file based on those parameters.
- Transfer the configuration to the gNB host using SFTP.
- Align the RU configuration to match the gNB setup.
- Update RU settings in Grafana.

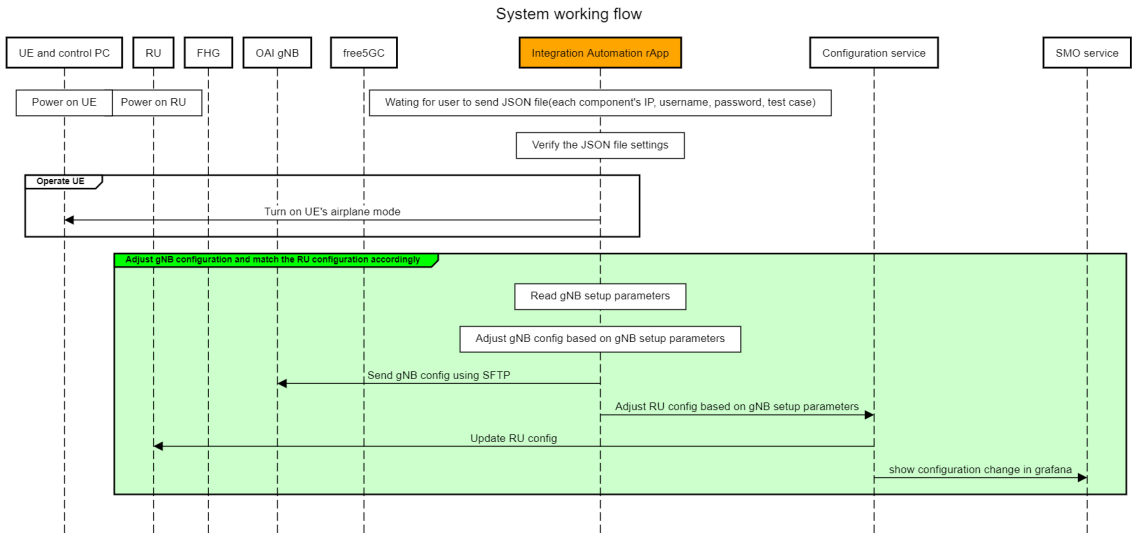

Figure 1.3: MSC for edit config of DU and RU

1.2.1 Repeatable parameters in DU/RU

As shown in Table 1.1, several configuration parameters—such as `l2_mtu`, `iq-bitwidth`, `compression-type`, and `compression-method`—are required to be consistently applied to both the DU and the RU.

Instead of manually duplicating these settings, we defined them once in a unified test case input. The integration rApp automatically parses this input and propagates the configuration values to both the DU and RU sides accordingly.

This repeatable configuration mechanism significantly reduces the manual effort and ensures consistency across layers, effectively cutting the configuration time in half for parameters that would otherwise require double input.

1.2.2 DU/RU MAC and VLAN Adjustment

In traditional manual setups, retrieving and configuring the MAC address of the RU requires manually logging into the terminal, executing network commands, and then writing the value into the configuration file. In our system, this process is automated. The ru id specified in Table 1.1 is used to automatically retrieve the RU MAC address and populate it into the configuration file via the rApp.

For the DU side, the manual process involves four main steps, as illustrated in Figure 1.4:

1. Identify the correct NIC (Network Interface Card) from which packets should be transmitted.
2. Create a VF (Virtual Function) [**?**] on the NIC and assign it a specific MAC address and VLAN tag.
3. Retrieve the VF's bus information and bind it to the DPDK (Data Plane Development Kit) [**?**].
4. Fill in the VF's bus information in the OAI gNB configuration file.

In our automated solution, all of the above steps are handled by the integration rApp. Users only need to provide the NIC interface name [**?**], as specified in the dpdk devices interface field of Table 1.1. The rApp will:

- Detect if a VF exists; if not, create it.
- Ensure the VF ' s MAC and VLAN match the test case.

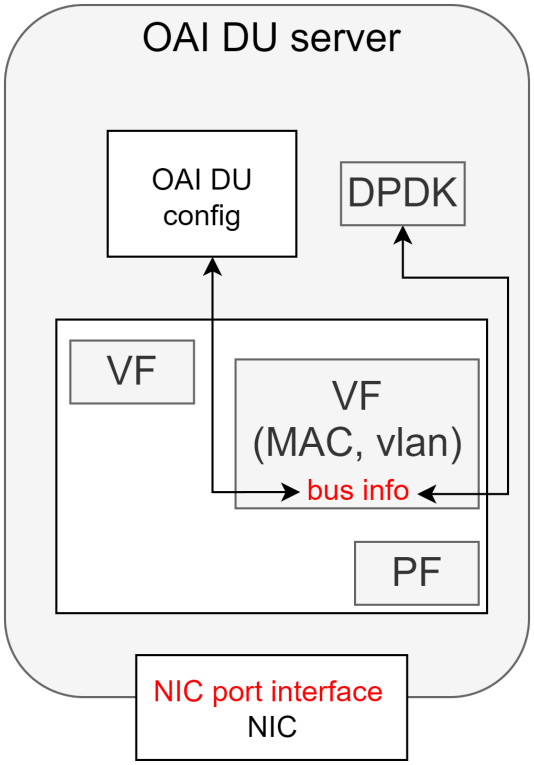

Figure 1.4: Manual configuration workflow for OAI gNB VF setup (MAC, VLAN, and binding).

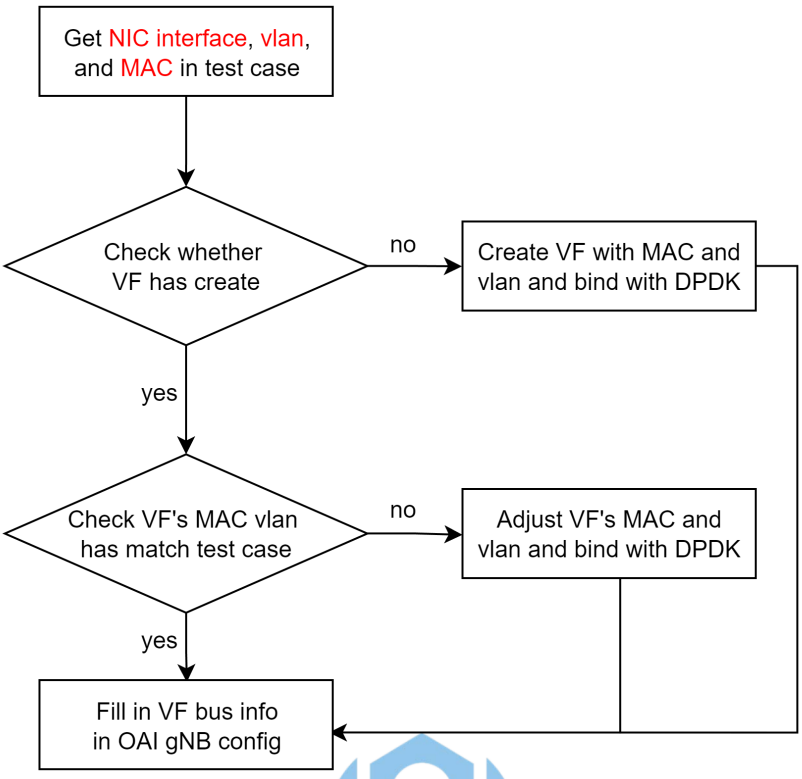

Figure 1.5: Flowchart for automatic create VF

- Bind the VF to DPDK using the correct PCI bus info.
- Automatically populate the OAI gNB config with this information.

This automation eliminates repetitive manual work and ensures consistency between the test case and system setup, as illustrated in Figure 1.5.

1.2.3 MIMO Adjustment

To support flexible downlink transmission configurations, the integration rApp automates the adjustment of MIMO (Multiple Input Multiple Output) settings according to the test case. As shown in Figure 1.6, the process begins by retrieving the antenna configuration and desired number

of MIMO layers from the test case specification. These values determine how many transmission layers should be active.

The rApp then generates the corresponding eAxC IDs to be filled into the OAI gNB configuration. If the specified number of MIMO layers exceeds the RU's antenna capability, the rApp automatically limits the layer count to the antenna ' s maximum capacity. Otherwise, it directly applies the test case value.

After finalizing the MIMO layer setting, the rApp maps each eAxC ID to the corresponding RU antenna and determines which antennas to activate or deactivate based on the mapping result. This ensures that the configuration aligns with both the test specification and the hardware ' s physical capabilities, enabling consistent and repeatable MIMO testing without requiring manual adjustment.

1.2.4 Frequency Adjustment

To ensure correct communication between DU and RU, several key parameters must be transformed or derived automatically from test input, as illustrated in Figures 1.8 and 1.9. The frequency configuration involves five major steps:

1. **SubCarrierSpacing (SCS) Conversion**: The input `subCarrierSpacing` (in kHz) is converted into a corresponding numerology value and an SCS identifier string (`scs`) for DU and RU configuration. This mapping follows the 3GPP standard numerology [**?**].

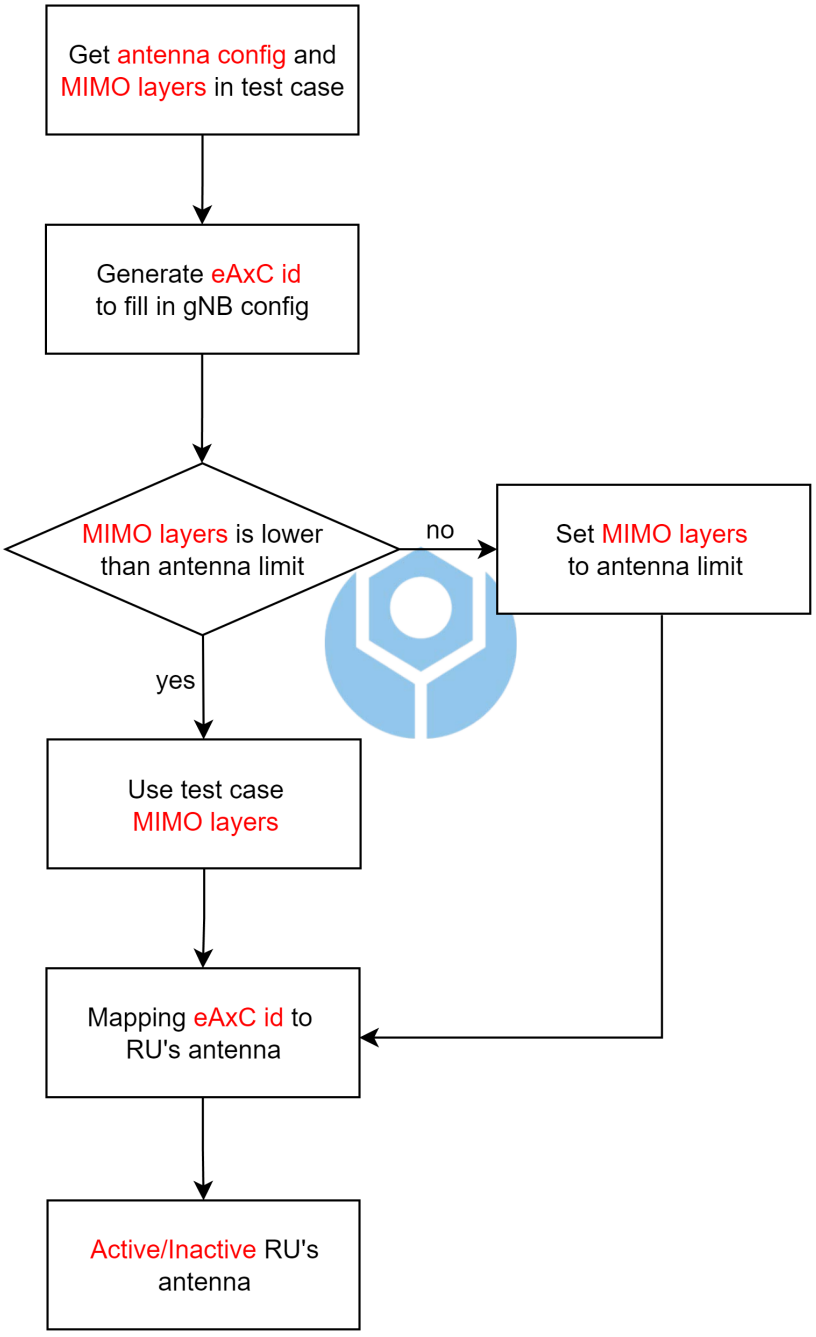

Figure 1.6: Flowchart for auto MIMO config

Table 1.4: SubCarrierSpacing Mapping

subCarrierSpacing (kHz)	Numerology	scs
15	0	`15_kHz`
30	1	`30_kHz`
60	2	`60_kHz`
120	3	`120_kHz`

2. **Bandwidth (BW) Conversion**: The test input provides the bandwidth (`bSChannelBw`) in MHz. This value is converted to the number of resource blocks (RBs) based on the selected subcarrier spacing (SCS), using the NRB mapping table defined in 3GPP TS 38.101-1 [**?**]. The resulting RB count is used to configure the DU's `carrierBandwidth`, as well as the RU's `number-of-prb` and `channel-bandwidth` parameters. Figure 1.7 shows the automatic conversion and mapping of the bandwidth setting to corresponding DU and RU parameters.

TS 3GPP TS 38.101-1, V15.2.0, Table 5.3.2-1: Maximum transmission bandwidth configuration NRB

SCS (kHz)	5MHz	10MHz	15MHz	20 MHz	25 MHz	30 MHz	40 MHz	50MHz	60 MHz	80 MHz	90 MHz	100 MHz
	N_{RB}	N_{RB}	N_{RB}	N_{RB}	N_{RB}	N_{RB}	N_{RB}	N_{RB}	N_{RB}	N_{RB}	N_{RB}	N_{RB}
15	25	52	79	106	133	160	216	270	N/A	N/A	N/A	N/A
30	11	24	38	51	65	78	106	133	162	217	245	273
60	N/A	11	18	24	31	38	51	65	79	107	121	135

Figure 1.7: Mapping between Bandwidth and Resource Block (RB) configuration for DU and RU

3. **SSB Frequency Selection**: The `ssbFrequency` in DU configuration is derived from the input `arfcn` using the following logic:

   - First, the absolute frequency (in Hz) corresponding to the input ARFCN is calculated.

- Then, the center of the channel bandwidth is determined using the equation:

$$f_{center} = f_{pointA} + \left(\frac{BW_{RB}}{2} \times SCS \times 12 \right)$$

- Based on this center frequency, the SSB frequency is selected by finding the closest point on the 3GPP-defined synchronization raster:

$$f_{SSB} = 3000 \text{ MHz} + N \times 1.44 \text{ MHz}$$

where $N = \left\lfloor \frac{f_{center} - 3000 \text{ MHz}}{1.44 \text{ MHz}} \right\rfloor$

- The corresponding `ssb_arfcn` is then computed and applied in DU

This aligns with the raster alignment rules defined in [**?**] for NR operating bands.

4. **RU Center Frequency Calculation**: The RU does not use ARFCN directly. Instead, its center frequency is computed using the number of PRBs (`number-of-prb`) and the same formula for center frequency:

$$f_{center}^{RU} = f_{pointA} + \left(\frac{PRB_{RU}}{2} \times SCS \times 12 \right)$$

This frequency is used for setting `center-of-channel-bandwidth` and `offset-to-absolute-frequency-center` parameters in the RU's M-plane configuration [**?**].

5. **BWP (Bandwidth Part) Location and Size**: DU configuration requires a compact encoding of the initial BWP location and size. This

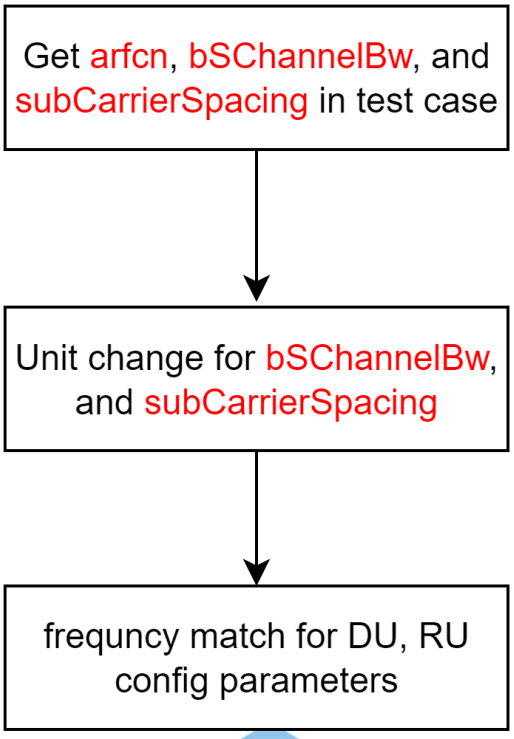

Figure 1.8: Flowchart for auto frequency config

is done via RIV (Resource Indication Value) encoding:

$$RIV = \begin{cases} N_{RB} \cdot (L-1) + R, & L \leq \frac{N_{RB}}{2} \\ N_{RB} \cdot (N_{RB} - L + 1) + (N_{RB} - 1 - R), & \text{otherwise} \end{cases}$$

where L is the BWP length (equal to BW in RB), R is the start RB index (set to 0), and $N_{RB} = 275$ is the maximum supported RB count. The encoded RIV is applied to both `initialDLBWPlocationAndBandwidth` and `initialULBWPlocationAndBandwidth` fields.

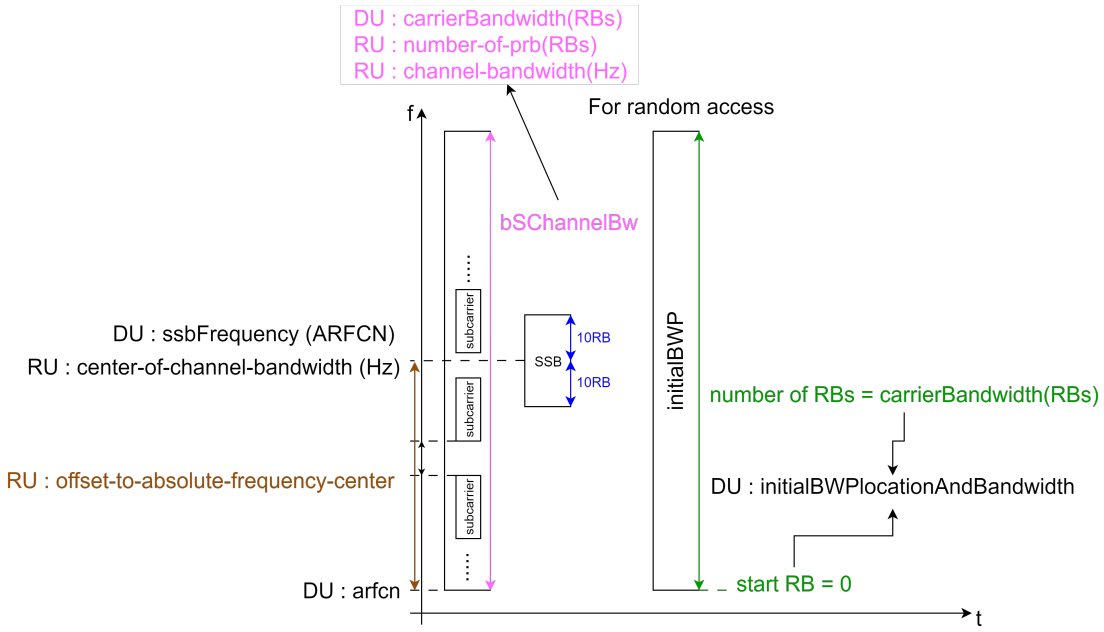

Figure 1.9: Frequenncy parameters in DU/RU config

1.3 Start End-to-end component

This section outlines the automated process for starting end-to-end (E2E) components in an O-RAN system, from DU/RU synchronization to gNB and Core Network (CN) activation and NG setup validation. Each step is monitored and verified by the Integration Automation rApp.

The flow begins with checking the synchronization state between DU and RU, as precise timing is required for proper fronthaul operation. Once synchronization is confirmed, the CN is started, followed by gNB activation.

Figure 1.11 shows the PTP synchronization verification process. The automation rApp continuously checks the synchronization state of the DU and RU by querying the gNB and M-plane interfaces. The retrieved status is stored in the test execution interface and evaluated to ensure readiness

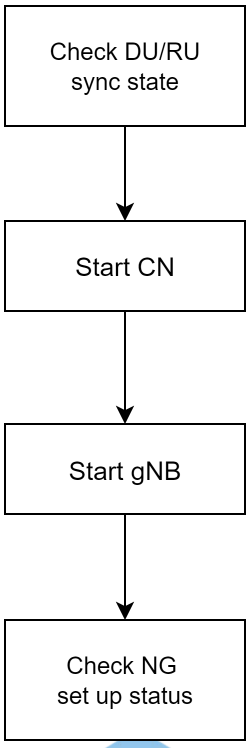

Figure 1.10: Flowchart of the end-to-end component start sequence

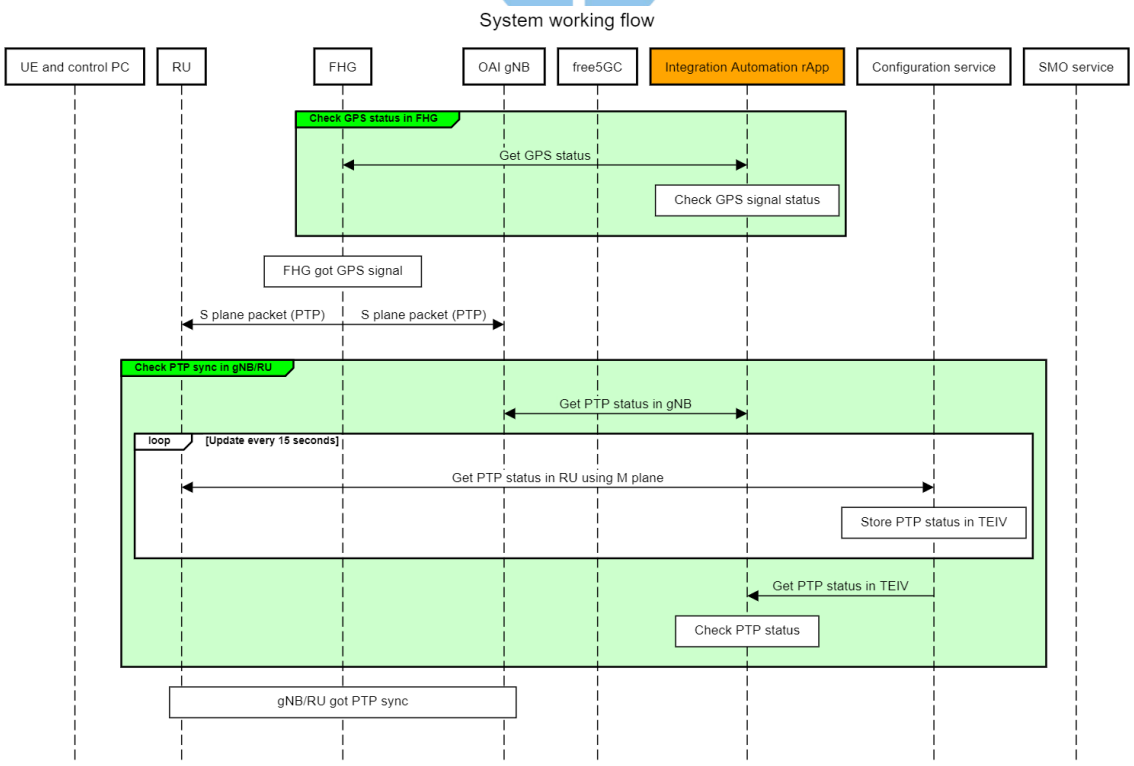

Figure 1.11: Message Sequence Chart for DU/RU PTP Synchronization Checking

before proceeding to the next steps. To determine if the DU is properly synchronized, the ‘ptp4l‘ logs are parsed to monitor key metrics such as `offset`, `delay`, and `frequency`. According to the guideline from the Time Sensitive Networking documentation [**?**], consistent and small offset/delay values (e.g., within ±1µs) indicate successful synchronization.

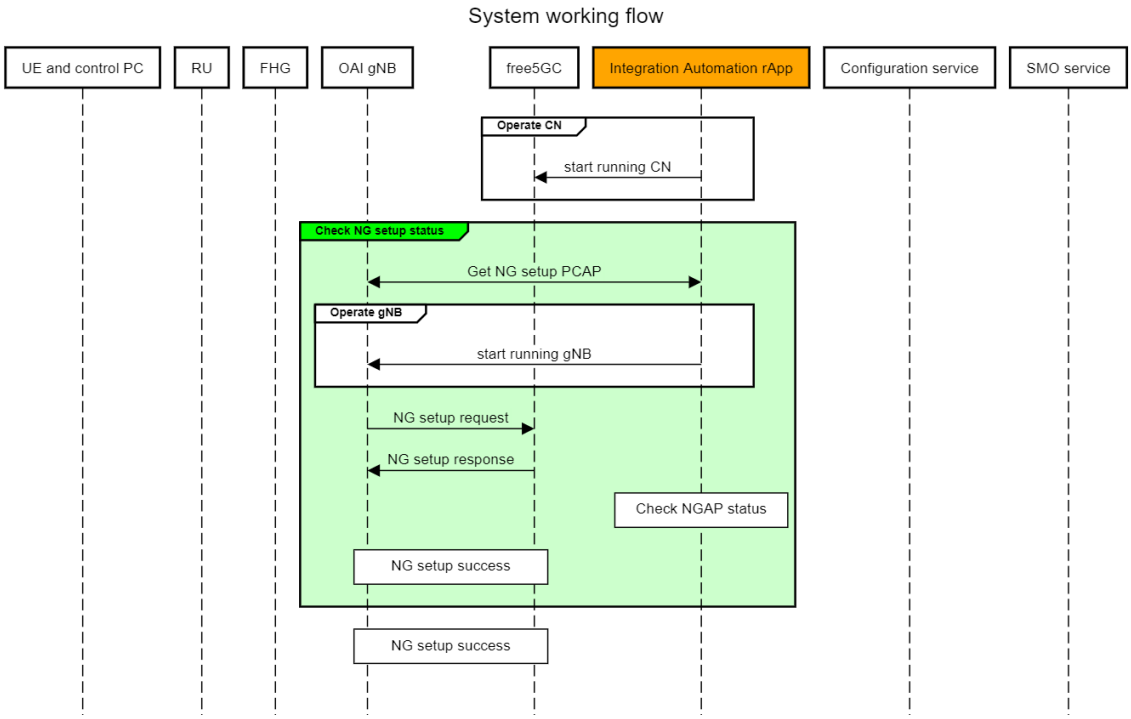

Figure 1.12: Message Sequence Chart and Flow for NG Setup Status Verification

After the CN and gNB are started, the Integration Automation rApp verifies whether the NG interface between the gNB and CN has been successfully established. As shown in Figure 1.12, this verification is performed by capturing and analyzing NGAP messages exchanged over the NG interface.

The automation tool captures the packet data using `tcpdump` [**?**] and analyzes the resulting PCAP files with `pyshark` [**?**], a Python wrapper for `tshark`. It specifically checks for the presence of an `NGSetupResponse`

following an `NGSetupRequest` to confirm successful setup of the NG interface.

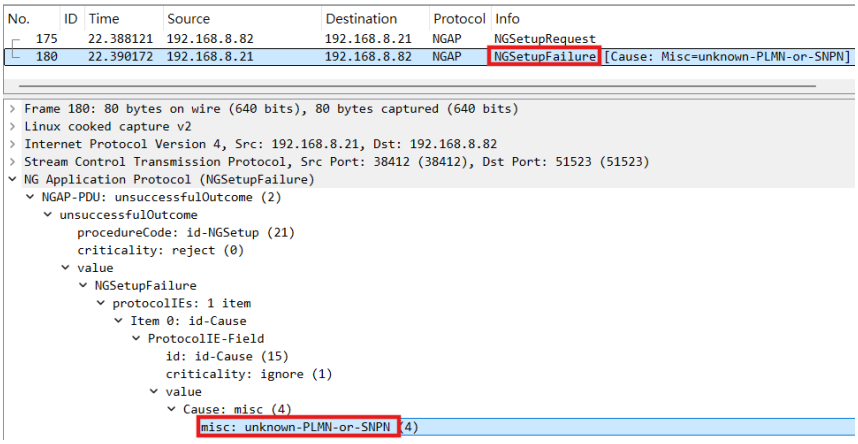

Figure 1.13: NG Setup Failure Packet Example: Cause = Unknown PLMN or SNPN

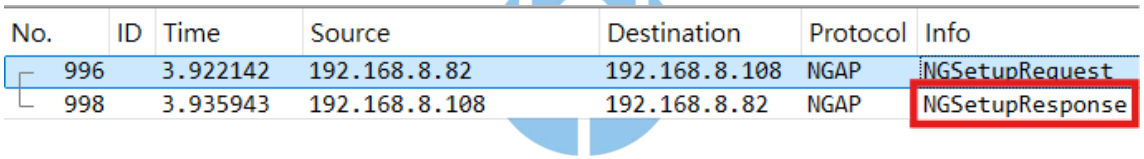

Figure 1.14: NG Setup Success Packet Example: NGSetupRequest followed by NGSetupResponse

Figures 1.13 and 1.14 provide concrete examples of NG setup failure and success, respectively. These diagnostic tools provide immediate feedback on configuration correctness and enable rapid identification of interoperability issues, thereby improving the reliability and repeatability of E2E integration testing.

1.4 Checking Fronthaul Packet Status and Timing Window Adjustment

To ensure reliable and timely transmission between the gNB and RU over the Open Fronthaul (FH) interface, the system must continuously monitor packet arrival patterns and make timing window adjustments accordingly. The integration rApp collects and analyzes packet status from the RU, focusing on whether C/U-plane packets fall within the configured timing window.

Figure 1.15 presents the overall procedure. Once the gNB is started, the rApp retrieves the RU's packet reception statistics from the configuration service and checks if packet timestamps fall within the expected window. If they do, the UE is instructed to proceed with attachment. Otherwise, the system automatically adjusts timing window parameters (T1a_cp and T1a_up) and restarts the gNB.

The logic for adjusting the timing window is illustrated in Figure 1.16. It checks for increasing trends in early or late packet counts (RX_EARLY or RX_LATE) and updates the corresponding T1a value to recenter packet timing into the target window.

Figures 1.17 and 1.18 show the visualization of how packets are categorized based on arrival time and how these statistics guide the adjustment process. Packets are classified as early, on-time, or late, and their trends determine whether the timing window should be widened or shifted.

The above procedure represents the method currently adopted in our lab-

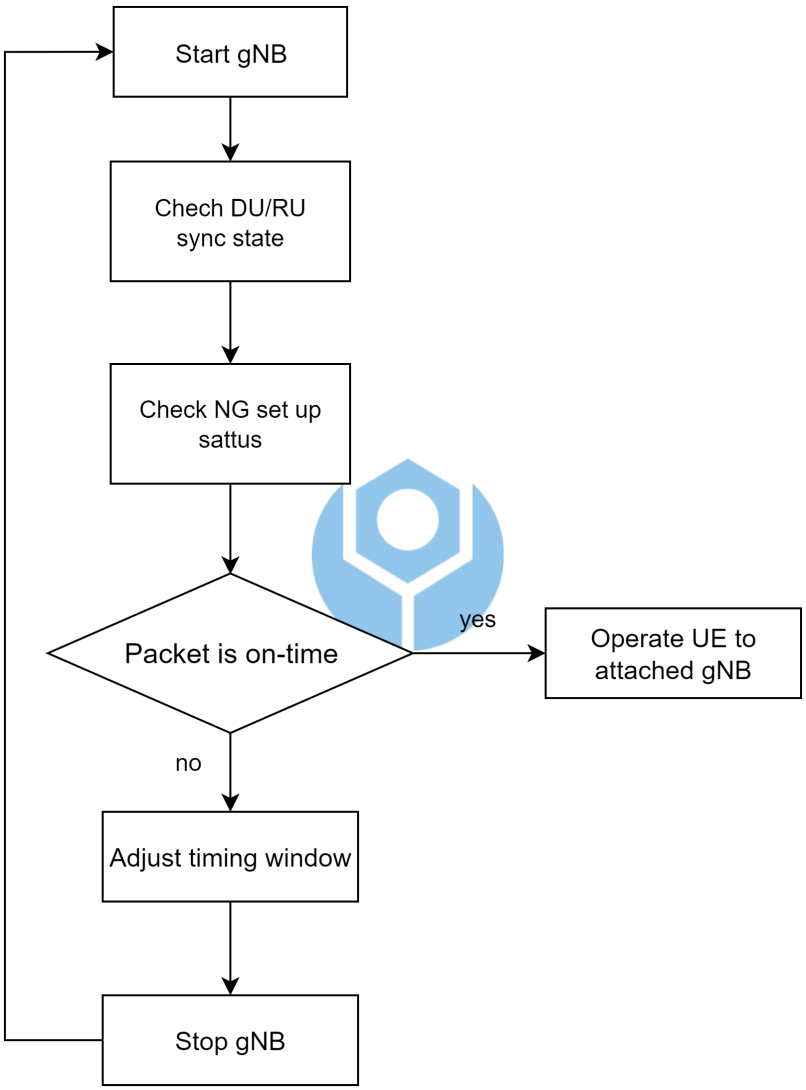

Figure 1.15: System architecture and flowchart for checking RU's C/U-plane packet timing status

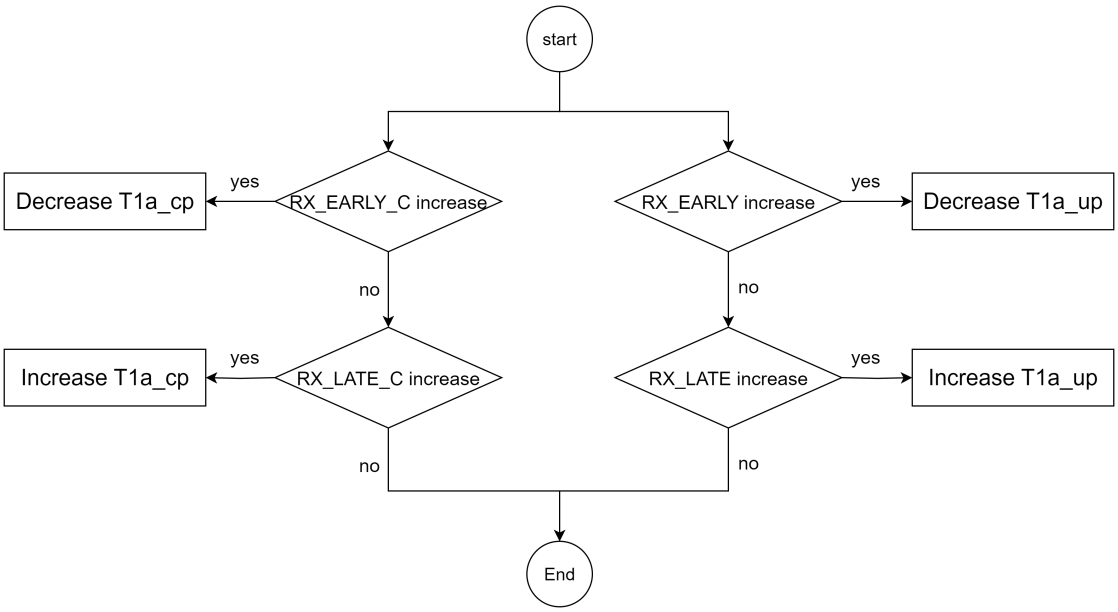

Figure 1.16: Flowchart for timing window adjustment based on RX_EARLY and RX_LATE statistics

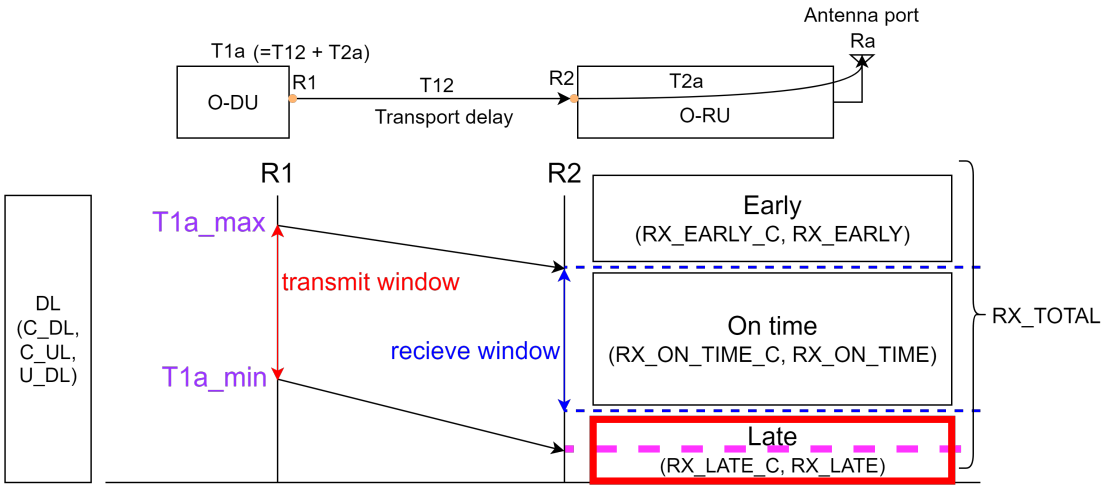

Figure 1.17: Timing window adjustment when RX_LATE increases, requiring an increase of T1a_cp or T1a_up

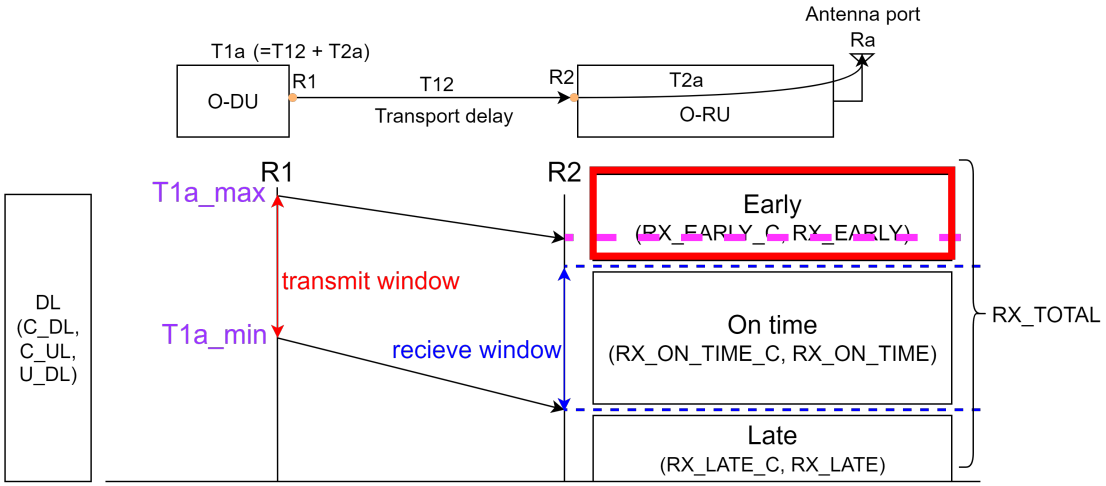

Figure 1.18: Timing window adjustment when RX_EARLY increases, requiring a decrease of T1a_cp or T1a_up

oratory, since the available NICs do not support advanced time-sensitive features. If high-end NICs were available, an alternative approach could be applied using **Time-Aware Shaper (TAS, IEEE 802.1Qbv)** [**?**], **global time synchronization (gPTP / PTP, IEEE 802.1AS)** [**?**], and the Linux `taprio` qdisc. These technologies enable deterministic packet scheduling and sub-microsecond synchronization, ensuring that fronthaul packets are automatically transmitted within the intended time slice. In fact, Al-Hares et al. demonstrated through modeling and simulation that TAS effectively eliminates frame delay variation in Ethernet fronthaul compared to traditional strict-priority scheduling, thereby validating the feasibility of this approach in meeting stringent timing requirements [**?**,**?**].

Such an approach would simplify the workflow by reducing manual operations across different stages:

- **Planning stage:** No need for repeated estimation of transmission timing to satisfy T1a, because advanced NICs with TAS can directly

schedule packet transmission at the specified time slice.

- **Testing stage:** Fewer iterative tests to verify timing parameters, because deterministic scheduling ensures that once configured, packets consistently meet the timing requirement without repeated tuning.
- **Verification stage:** Reduced manual verification of packet timestamps due to hardware timestamping and deterministic scheduling, which guarantee precise measurement and compliance with the configured timing window.

1.5 UE Auto Attach and iperf

This section describes the automated procedure for connecting the User Equipment (UE) to the gNB and performing throughput testing using `iperf3`. The entire process is fully automated using `adb` commands and backend integration, enabling the system to determine when the UE is attached and ready for performance testing.

Figure 1.19 shows the automated workflow: (1) turn off UE's airplane mode, (2) obtain the UE IP address, (3) start the `iperf3` server on the core network (CN), (4) run the `iperf3` client on the UE, and (5) collect and visualize throughput results.

Figure 1.20 illustrates the `iperf3` setup [**?**]. The UE acts as the client, sending data toward the CN. Parameters such as target bitrate and test duration can be configured, enabling reproducible throughput measurements.

Figure 1.21 shows how airplane mode on Android UE is controlled

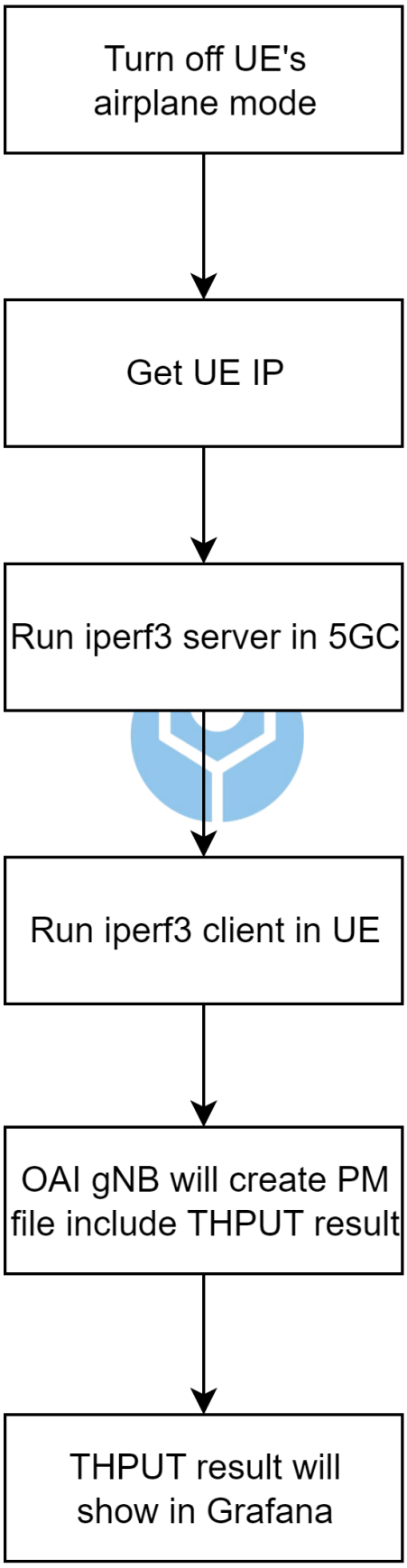

Figure 1.19: Automated UE attachment and iperf3 testing workflow.

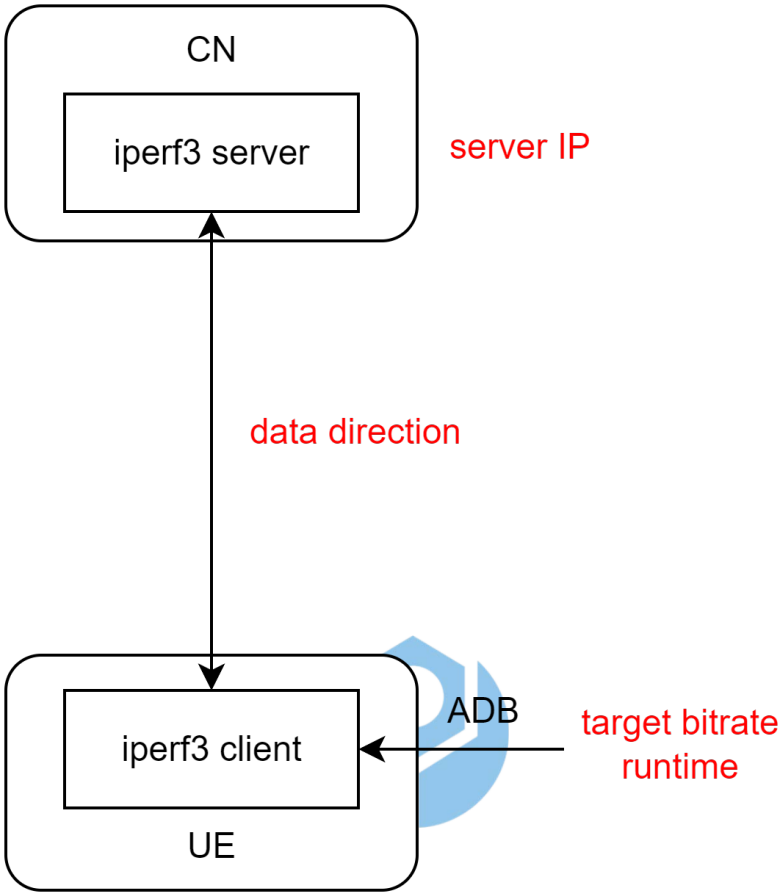

Figure 1.20: iperf3 client-server configuration.

Disable airplane mode:

```
PS C:\Users\bmwla> adb shell settings put global airplane_mode_on 0
PS C:\Users\bmwla> adb shell am broadcast -a android.intent.action.AIRPLANE_MODE --ez state false
Broadcasting: Intent { act=android.intent.action.AIRPLANE_MODE flg=0x400000 (has extras) }
Broadcast completed: result=0
```

Enable airplane mode:

```
PS C:\Users\bmwla> adb shell settings put global airplane_mode_on 1
PS C:\Users\bmwla> adb shell am broadcast -a android.intent.action.AIRPLANE_MODE --ez state true
Broadcasting: Intent { act=android.intent.action.AIRPLANE_MODE flg=0x400000 (has extras) }
Broadcast completed: result=0
```

Figure 1.21: ADB commands to toggle UE airplane mode.

via ADB. The automation system toggles this to reset radio connectivity before attachment.

No attahced gNB:

```
PS C:\Users\bmwla> adb shell ip -f inet addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
```

Attahced gNB:

```
PS C:\Users\bmwla> adb shell ip -f inet addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
2: ccmni0: <NOARP,UP,LOWER_UP> mtu 1400 qdisc mq state UNKNOWN group default qlen 1000
    inet 10.60.0.6/8 scope global ccmni0
       valid_lft forever preferred_lft forever
```

Figure 1.22: IP configuration differences before and after UE attachment.

Figure 1.22 compares the UE's IP address states. When unattached, only the loopback interface is present. After attachment to the gNB, a new interface (e.g., `ccmni0`) appears with an assigned IP address.

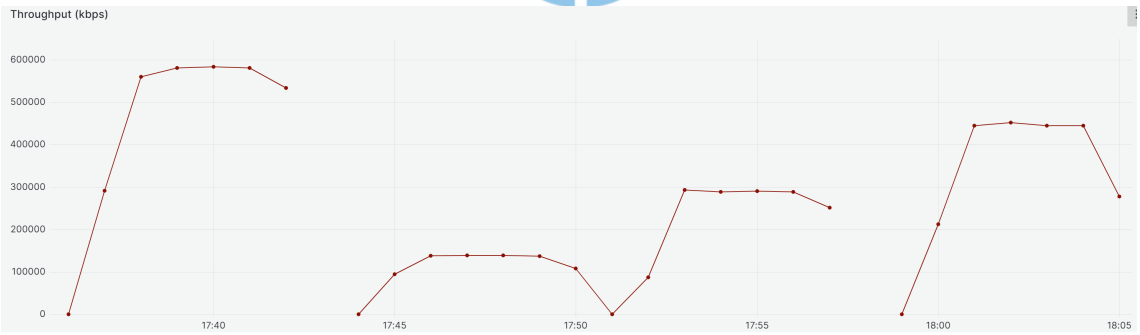

Figure 1.23: Grafana visualization of THPUT results extracted from OAI PM logs.

Once the iperf test is completed, the OAI gNB generates a performance management (PM) file [**?**] that includes the throughput (THPUT) results. As shown in Figure 1.23, these results are parsed and displayed on Grafana in near real-time, enabling intuitive performance monitoring.